

Allzweck-GPU- Programmierung mit CUDA

Vorlesung 2: Performance – den
Kernel schnell machen

Torsten Bosse

6. Juli 2026

Lehrstuhl für Advanced Computing





Rückblick auf Vorlesung 1

- GPUs sind **Durchsatz-Maschinen**; Speicherlatenz wird durch viele residente Warps verdeckt.
- Hierarchie *Thread* → *Warp* → *Block* → *Grid*, vom Scheduler auf die SMs abgebildet.
- Ein **Kernel** ersetzt eine datenparallele Schleife; jeder Thread berechnet seinen globalen Index selbst.
- Der Host orchestriert: allokieren, kopieren, starten, zurückkopieren, freigeben.
- Speicherhierarchie: Register → Shared Memory → globaler Speicher (schnell/klein → langsam/groß).

Letztes Mal: korrekter Code. Heute: effizienter Code.



Fahrplan dieser Vorlesung

Von „*Wodurch ist ein Kernel begrenzt?*“ zur Optimierung:

- 1 **Roofline** – speicher- oder rechengebunden?
- 2 **Globaler Speicher** – Coalescing, Datenlayout (SoA).
- 3 **SIMT & Divergenz** – der Preis von Verzweigungen (ab Volta).
- 4 **Shared Memory** – Wiederverwendung; Bankkonflikte.
- 5 **Occupancy** – Latenz verdecken.
- 6 **Warp-Primitive & Atomics** – effiziente Kooperation.
- 7 **Fallstudie:** gekachelte Matrixmultiplikation.
- 8 **Streams, Bibliotheken, Profiling.**



- Kernel mit dem *Roofline-Modell* als speicher- oder rechengebunden einordnen;
- globale Zugriffe *coalesced* gestalten und Datenlayout (AoS vs. SoA) bewerten;
- *Shared Memory* zur Datenwiederverwendung einsetzen + *Bankkonflikte* vermeiden;
- *Occupancy*, *Warp-Divergenz* und Warp-Primitive/Atomics richtig einordnen;
- Überlappen von Daten-Transfer und Berechnungen mittels *Streams*
- *Bibliotheken* und *Profiler* nutzen.

Performance-Denken: das Roofline-Modell

GPGPU / CUDA – Vorlesung 2



Das Roofline-Modell

Praktisches visuelles Modell für die Obergrenze für erreichbare Leistung!

■ Arithmetische (operationale) Intensität

$$I = \frac{\text{FLOPs}}{\text{aus dem DRAM bewegte Bytes}} \quad [\text{FLOP/Byte}].$$

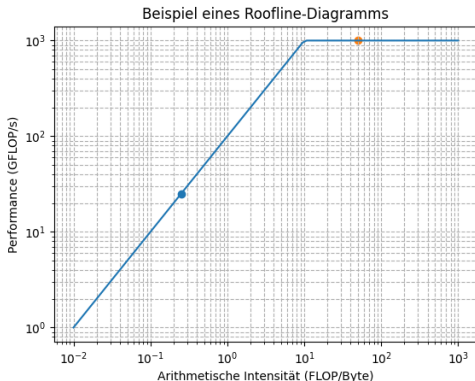
■ Erreichbare Leistung **Performance**:

$$P \leq \min(P_{\text{peak}}, B_{\text{peak}} \cdot I)$$

wobei P_{peak} die *Peak-Performance* (GFLOP/s) und B_{peak} die *Peak-Bandwidth* (GB/s) ist.



Das Roofline-Diagramm



- **Knickpunkt (ridge point):** $I^* = P_{\text{peak}}/B_{\text{peak}}$ trennt zwei Bereiche.
- Links des Knicks: **speichergebunden** (memory-bound) – Datenbewegung optimieren (Coalescing, Wiederverwendung, Shared Memory).
- Rechts des Knicks: **rechengebunden** (compute-bound) – die Rechenoperationen optimieren (Instruktionsmix, Tensor-Cores).



Berechnete Intensität: SAXPY

$$y_i \leftarrow a x_i + y_i$$

- Pro Element: 2 FLOPs (eine Multiplikation, eine Addition).
- Speicher: lese x_i , lese y_i , schreibe $y_i = 3$ Wörter = 12 Byte (FP32).
- $I = 2/12 \approx 0,17$ FLOP/Byte $\Rightarrow$ tief im speichergebundenen Bereich.

Keine Rechen-Optimierung hilft; nur das *Reduzieren der bewegten Bytes*. Verschmelzen von Operationen (siehe `saxpy_functor` bei Thrust) reduziert Lese- und Schreibzugriffe.

Globaler Speicher & Coalescing

GPGPU / CUDA – Vorlesung 2



Coalescing globaler Speicherzugriffe

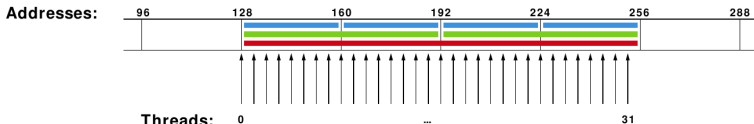
Grid-Stride begünstigt *coalesced* Zugriffe im Detail:

- Auf den globalen Speicher wird jeweils durch einen *Warp* (32 Threads) zugegriffen.
- Die Hardware bedient den Speicher in ausgerichteten 32/64/128-Byte-Transaktionen.
- *Coalesced*: die 32 Threads treffen ein (oder wenige) zusammenhängende, ausgerichtete Segment(e)
⇒ minimale Anzahl Transaktionen, volle Bandbreite.
- *Uncoalesced*: gestreute oder strided Zugriffe
⇒ viele Transaktionen, verschwendete Bandbreite.
- *Faustregel*: Thread t sollte auf Element $\text{base} + t$ zugreifen (Schrittweite 1 über den Warp).



Coalescing – die drei Bilder (1/3)

Aligned and sequential



Compute capability:	1.0 and 1.1	1.2 and 1.3	2.x and 3.x	
Memory transactions:	Uncached		Uncached	Cached
	1x 64B at 128 1x 64B at 192	1x 64B at 128 1x 64B at 192	1x 32B at 128 1x 32B at 160 1x 32B at 192 1x 32B at 224	1x 128B at 128

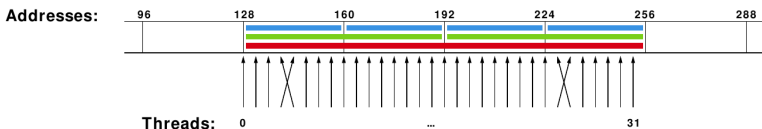
Aufeinanderfolgende Threads lesen aufeinanderfolgende Adressen
 $\Rightarrow$ eine Transaktion (ideal).

<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>



Coalescing – die drei Bilder (2/3)

Aligned and non-sequential



Compute capability:	1.0 and 1.1	1.2 and 1.3	2.x and 3.x	
Memory transactions:	Uncached		Uncached	Cached
	8x 32B at 128 8x 32B at 160 8x 32B at 192 8x 32B at 224	1x 64B at 128 1x 64B at 192	1x 32B at 128 1x 32B at 160 1x 32B at 192 1x 32B at 224	1x 128B at 128

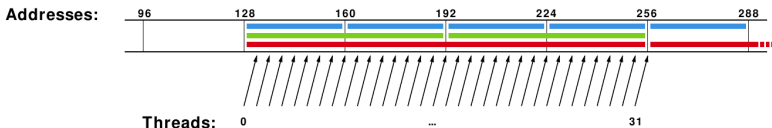
Eine Permutation innerhalb eines ausgerichteten Segments ist unproblematisch auf modernen GPUs $\Rightarrow$ weiterhin eine Transaktion.

<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>



Coalescing – die drei Bilder (3/3)

Mis-aligned and sequential



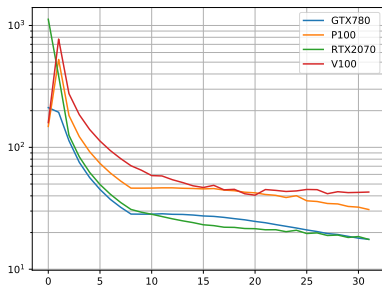
Compute capability:	1.0 and 1.1	1.2 and 1.3	2.x and 3.x	
Memory transactions:	Uncached		Uncached	Cached
	7x 32B at 128	1x 128B at 128	1x 32B at 128	1x 128B at 128
	8x 32B at 160	1x 64B at 192	1x 32B at 160	1x 128B at 256
	8x 32B at 192	1x 32B at 256	1x 32B at 192	
	8x 32B at 224		1x 32B at 224	
	1x 32B at 256		1x 32B at 256	

Fehlausgerichtete oder strided Zugriffe überspannen Segmente
 ⇒ zusätzliche Transaktionen, geringere effektive Bandbreite. Genau deshalb unterscheiden sich Ax und $A^T x$ in der Performance.

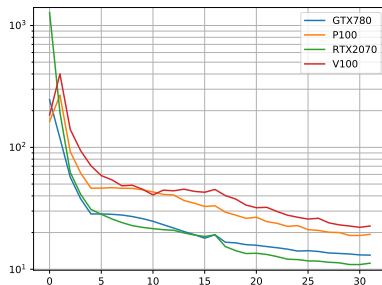


Gemessen: Bandbreite über der Schrittweite

FP32



FP64



Strided Zugriff ($a[\text{stride} \cdot i]$) bricht die effektive Bandbreite ein: bei Schrittweite 1 voll coalesced, mit wachsender Schrittweite immer mehr verschwendete Transaktionen.



Datenlayout: AoS vs. SoA

Array of Structs (AoS)

```
struct P { float x, y, z; };  
P part[N];  
// Thread t liest part[t].x  
// -> x-Werte liegen mit  
// Schritt 3 -> strided!
```

Struct of Arrays (SoA)

```
struct P { float x[N], y[N],  
           z[N]; };  
P part;  
// Thread t liest part.x[t]  
// -> zusammenhaengend  
// -> coalesced!
```

Auf der GPU ist **SoA** fast immer das bessere Layout: Threads eines Warps greifen auf benachbarte Elemente *desselben* Feldes zu. Dasselbe Datum, eine kleine Layout-Entscheidung – großer Bandbreitenunterschied.

SIMT & Sprung-Divergenz

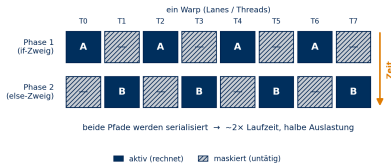
GPGPU / CUDA – Vorlesung 2



Divergenz (Wiederholung)

Ein Warp führt eine Instruktion im Gleichschritt aus (SIMT). Nehmen die Threads an einem `if/else` verschiedene Pfade, werden diese *serialisiert* und die Auslastung sinkt.

- Datenabhängige Verzweigungen im Warp minimieren.
- Sprungbedingungen möglichst an Warp-Grenzen (Vielfache von 32) ausrichten.
- **Neue Frage:** Wann laufen die Pfade wieder zusammen – und können Threads desselben Warps *unabhängig* Fortschritt machen?

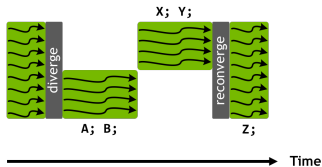


Branch Divergence



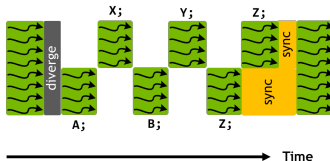
■ pre-Volta:

```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```



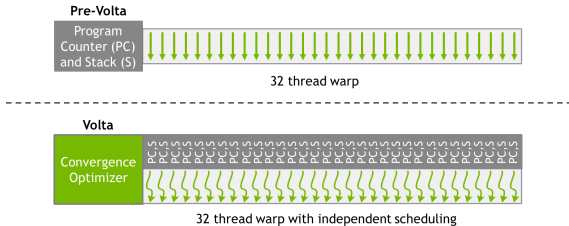
■ Volta und später:

```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;  
__syncwarp();
```



<https://developer.nvidia.com/blog/inside-volta/>

Branch Divergence – Warp-Scheduling



<https://developer.nvidia.com/blog/inside-volta/>

Shared Memory

GPGPU / CUDA – Vorlesung 2



Shared Memory

- Threads im *selben Block* tauschen Daten über On-Chip-**Shared Memory** aus – einen software-verwalteten Scratchpad.
- Deklariert mit `__shared__`; statisch im Kernel oder dynamisch beim Start dimensioniert.
- Um Größenordnungen niedrigere Latenz als globaler Speicher.
- Nach Schreiben in Shared Memory Block mit `__syncthreads()` synchronisieren, bevor andere Threads lesen.
- Klassische Anwendung: eine Kachel globaler Daten einmal *einlagern* und dann vielfach aus dem Shared Memory wiederverwenden.



Shared Memory – Block-Reduktion

```
__global__ void smreduce(const float *a, float *b) {  
    int tid = threadIdx.x;  
    int id = tid + blockIdx.x * blockDim.x;  
    __shared__ float sm[512];  
    sm[tid] = a[id];  
    __syncthreads();  
    for (int s = blockDim.x / 2; s > 0; s /= 2) {  
        if (tid < s) sm[tid] += sm[tid + s];  
        __syncthreads();           // jeder Thread muss hier ankommen  
    }  
    if (tid == 0) b[blockIdx.x] = sm[0];  
}
```

Eine Baum-Reduktion in $\log_2(\text{blockDim})$ Schritten; die Teilsummen bleiben On-Chip.

Bankkonflikte

GPGPU / CUDA – Vorlesung 2

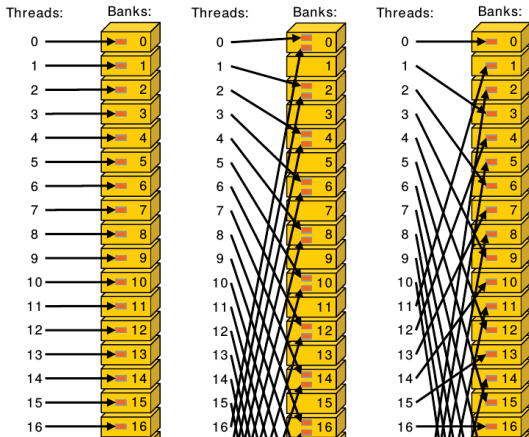


Shared-Memory-Bänke

- Shared Memory ist in **32 Bänke** aufgeteilt.
- Aufeinanderfolgende 32-Bit-Wörter liegen in aufeinanderfolgenden Bänken (Wort $w \rightarrow \text{Bank } w \bmod 32$).
- Eine Bank liefert (grob) ein 32-Bit-Wort pro Takt.
- **Bankkonflikt:** zwei oder mehr Threads eines Warps greifen auf *verschiedene Wörter derselben* Bank zu
 $\Rightarrow$ die Zugriffe werden serialisiert.
- Ausnahme: Lesen alle Threads *dasselbe* Wort, so führt die Hardware einen *Broadcast* aus – kein Konflikt.



Bankkonflikte – strided Zugriff

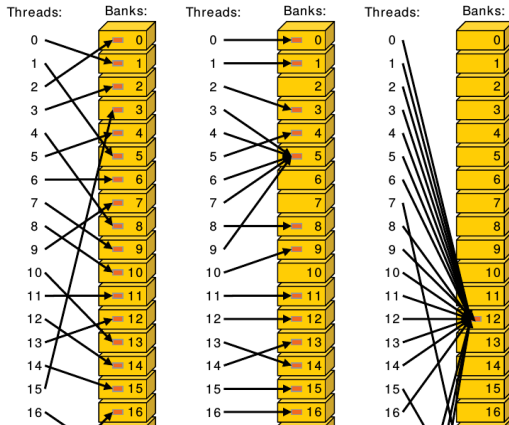


links: konfliktfrei, Mitte: 2-fach Bankkonflikte, rechts: konfliktfrei

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



Bankkonflikte – Broadcast



Lesen viele Threads *dieselbe* Adresse, ist dies dank des Broadcast-Mechanismus seit jeher konfliktfrei.

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



Abhilfe: Padding

Beispiel: ein quadratisches Shared-Memory-Tile, das spaltenweise gelesen wird, erzeugt 32-fache Konflikte (alle treffen dieselbe Bank).

```
__shared__ float tile[32][32]; // Spaltenzugriff -> 32-Wege-Konflikt  
__shared__ float tile[32][32 + 1]; // +1 Padding -> konfliktfrei
```

Eine zusätzliche Spalte verschiebt jede Zeile um eine Bank, sodass eine Spalte über alle 32 Bänke verteilt liegt. Kosten: minimal mehr Shared Memory; Nutzen: keine Serialisierung.

Occupancy

GPGPU / CUDA – Vorlesung 2



Occupancy (Belegung)

$$\text{Occupancy} = \frac{\text{aktive Warps pro SM}}{\text{maximale Warps pro SM}}$$

- Latenz wird verdeckt, indem *viele* residente Warps zum Umschalten bereitstehen – so toleriert die GPU Speicherlatenz von mehreren hundert Takten.
- Die Zahl residenter Blöcke pro SM ist durch die knappste Ressource begrenzt:
 - Register pro Thread,
 - Shared Memory pro Block,
 - Hardware-Limits für Warps/Blöcke.
- Hohe Occupancy hilft, Latenz zu verdecken – ist aber ein *Mittel*, kein Ziel. Ein Kernel mit niedriger Occupancy, aber genug Instruktionsparallelität, kann die Hardware ebenfalls auslasten.

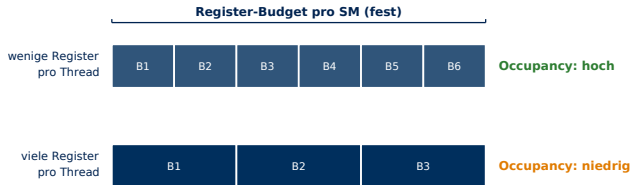


Occupancy – Beispiel

```
1  #define BLOCK_DIM 256
2  __global__ void kernel(int *a, int *b, int size){
3      int tid = threadIdx.x;
4      __shared__ int s_buff[BLOCK_DIM];
5      s_buff[tid] = a[tid+blockDim.x*blockIdx.x];
6      __syncthreads();
7      for (int i = BLOCK_DIM / 2; i > 0; i /= 2) {
8          if (tid < i)
9              s_buff[tid] += s_buff[tid+i];
10         __syncthreads();
11     }
12     if (tid == 0)
13         atomicAdd(&b[0], s_buff[0]);
14 }
15 int main(int argc, char**argv)
16 {
17     int SIZE=1024;
18     int *a_host, *b_host, *a_dev, *b_dev;
19     a_host = (int*)malloc(SIZE*sizeof(int));
20     for (int i=0;i<SIZE;i++)
21         a_host[i]=1;
22     b_host = (int*)malloc(SIZE*sizeof(int));
23     fillA(a_host,SIZE);
24     cudaMalloc((void**)&a_dev,SIZE*sizeof(int));
25     cudaMalloc((void**)&b_dev,SIZE*sizeof(int));
26     cudaMemcpy(a_dev,a_host,SIZE*sizeof(int),cudaMemcpyHostToDevice);
27     kernel<<<(SIZE+BLOCK_DIM-1)/BLOCK_DIM,BLOCK_DIM>>>>(a_dev,b_dev,SIZE);
28     cudaMemcpy(b_host,b_dev,SIZE*sizeof(int),cudaMemcpyDeviceToHost);
29     checkResult(a_host,b_host,SIZE);
30     cudaFree(a_dev);
31     cudaFree(b_dev);
32     free(a_host);
33     free(b_host);
34     return 0;
35 }
```



Occupancy – der Zielkonflikt



Mehr Register/Thread (oder Shared Memory/Block) → weniger residente Blöcke pro SM.

Mit `-Xptxas -v` die Register pro Thread anzeigen und mit dem CUDA Occupancy Calculator / `cudaOccupancyMaxActiveBlocksPerMultiprocessor` argumentieren.

Warp-Primitive & Atomics

GPGPU / CUDA – Vorlesung 2



Warp-Shuffle: Reduktion ohne Shared Memory

```
__inline__ __device__ float warpReduceSum(float val) {  
    for (int offset = 16; offset > 0; offset /= 2)  
        val += __shfl_down_sync(0xffffffff, val, offset);  
    return val;    // Lane 0 enthaelt die Summe des Warps  
}
```

- `__shfl*_sync` tauscht Register direkt zwischen den Lanes eines Warps aus
– ohne Shared Memory, ohne `__syncthreads()`.
- Die Maske `0xffffffff` benennt die teilnehmenden Lanes (hier: alle).



Atomare Operationen

- Unteilbares Read-Modify-Write über Threads hinweg: `atomicAdd`, `atomicMax`, `atomicCAS`, ...
- Nützlich für Histogramme, Reduktionen auf eine einzelne Stelle, Zähler.
- Achtung: Gleitkomma-`atomicAdd` ist *nicht assoziativ* $\Rightarrow$ Ergebnisse können sich in den letzten Bits von Lauf zu Lauf unterscheiden.

```
__global__ void maxArray(const int *arr, int *maxVal, int size) {  
    int id = threadIdx.x + blockIdx.x * blockDim.x;  
    if (id < size) atomicMax(maxVal, arr[id]);  
}
```



Beispiel: Histogramm mit Atomics

```
__global__ void histogram(const unsigned char *data, int n,
                          unsigned int *bins) {
    int i      = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (; i < n; i += stride)
        atomicAdd(&bins[data[i]], 1u);    // viele Threads, ein Bin
}
```

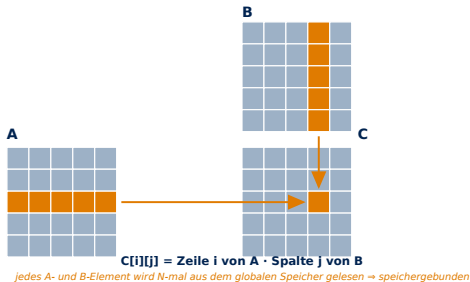
- Korrekt dank Atomarität – aber *Contention*, wenn viele Threads dieselben Bins treffen.
- Optimierung: pro Block ein *privates* Histogramm im Shared Memory aufbauen, am Ende einmal in das globale addieren.

Fallstudie: Tiled Matrixprodukt

GPGPU / CUDA – Vorlesung 2



Naive Matrixmultiplikation



Jedes Ausgabeelement liest erneut eine ganze Zeile von A und Spalte von B aus dem globalen Speicher $\Rightarrow$ geringe arithmetische Intensität, speichergebunden.

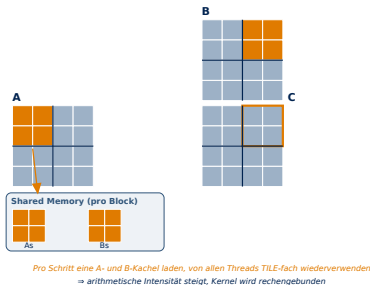


Naive Matrixmultiplikation – Kernel

```
__global__ void MatMul(const float *A, const float *B,
                      float *C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float acc = 0.0f;
        for (int k = 0; k < N; ++k)
            acc += A[row*N + k] * B[k*N + col];
        C[row*N + col] = acc;
    }
}
```



Gekachelte Matrixmultiplikation (Shared Memory)



Eine Kachel von A und B in den Shared Memory einlagern, jedes geladene Element `BLOCK_SIZE`-mal wiederverwenden $\Rightarrow$ die Intensität steigt, der Kernel bewegt sich Richtung rechengebunden.



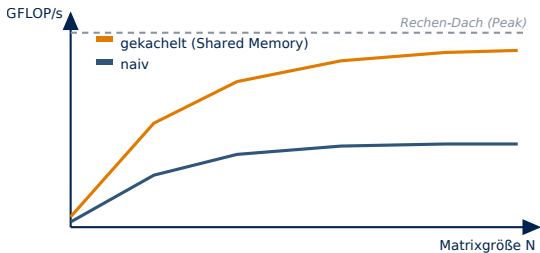
Gekachelter Kernel (Kernschleife)

```
__global__ void MatMulTiled(const float *A, const float *B,
                           float *C, int N) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];
    int row = blockIdx.y*TILE + threadIdx.y;
    int col = blockIdx.x*TILE + threadIdx.x;
    float acc = 0.0f;
    for (int m = 0; m < N/TILE; ++m) {
        As[threadIdx.y][threadIdx.x] = A[row*N + (m*TILE + threadIdx.x)];
        Bs[threadIdx.y][threadIdx.x] = B[(m*TILE + threadIdx.y)*N + col];
        __syncthreads(); // Kachel geladen
        for (int k = 0; k < TILE; ++k)
            acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads(); // vor dem Neuladen
    }
    C[row*N + col] = acc;
}
```

Vollständiger Code: CUDA C++ Programming Guide, Shared Memory.



Naiv vs. gekachelt – erwartetes Ergebnis



Wiederverwendung schiebt den Kernel auf der Roofline nach oben

Wiederverwendung über Shared Memory erhöht die arithmetische Intensität und schiebt den Kernel auf seiner Roofline nach oben in Richtung Rechen-Dach.

Überlappen: Streams & asynchrone Kopien

GPGPU / CUDA – Vorlesung 2



Der versteckte Engpass: PCIe-Transfers

- Datenaustausch zwischen Host $\leftrightarrow$ Device läuft über PCIe/NVLink und ist oft *langsamer* als der Kernel selbst.
- Standard-`cudaMemcpy` ist **synchron**: kopieren – rechnen – zurückkopieren laufen streng nacheinander.
- Idee: **überlappen** – während Kachel k rechnet, wird Kachel $k+1$ schon kopiert.
- Voraussetzung: **Pinned (page-locked) Host-Speicher** (`cudaMallocHost`) – nur dann sind Kopien asynchron.



Streams – Kopieren und Rechnen überlappen

```
cudaStream_t s[2];
cudaStreamCreate(&s[0]); cudaStreamCreate(&s[1]);
for (int c = 0; c < chunks; ++c) {
    int k = c & 1;                                // Stream abwechseln
    cudaMemcpyAsync(d+off, h+off, bytes,
                   cudaMemcpyHostToDevice, s[k]);
    kernel<<<grid, threads, 0, s[k]>>>(d+off, ...);
    cudaMemcpyAsync(h+off, d+off, bytes,
                   cudaMemcpyDeviceToHost, s[k]);
}
```

Operationen in *verschiedenen* Streams dürfen gleichzeitig laufen; in *einem* Stream bleibt die Reihenfolge erhalten. So füllt sich die Zeitachse: Kopier- und Rechen-Engine arbeiten parallel.

Bibliotheken

GPGPU / CUDA – Vorlesung 2



Das Rad nicht neu erfinden: CUDA-Bibliotheken

- **cuBLAS** – dichte lineare Algebra (GEMM, GEMV, ...), stark optimiert.
- **cuSPARSE**, **cuSOLVER**, **cuFFT**, **cuRAND** – dünnbesetzt, Löser, FFT, Zufallszahlen.
- **Thrust** – STL-ähnliche parallele Algorithmen und Container (`device_vector`, `transform`, `reduce`, `sort`, **Scans**).
- **CUB** – Bausteine (Block-/Warp-Reduktionen, Scans).
- *Faustregel*: vor dem eigenen Kernel zur Bibliothek greifen; cuBLAS für GEMM schlägt man selten.



Thrust – verschmolzenes SAXPY

```
struct saxpy_functor {  
    const float a;  
    saxpy_functor(float a_) : a(a_) {}  
    __host__ __device__  
    float operator()(const float &x, const float &y) const {  
        return a*x + y;  
    }  
};  
  
void saxpy_fast(float a, thrust::device_vector<float> &X,  
               thrust::device_vector<float> &Y) {  
    thrust::transform(X.begin(), X.end(), Y.begin(),  
                     Y.begin(), saxpy_functor(a));  
}
```

Verschmelzen zu einem einzigen `transform` senkt den Speicherverkehr.



cuBLAS – GEMM in wenigen Zeilen

```
cublasHandle_t h;  cublasCreate(&h);  
const float alpha = 1.0f, beta = 0.0f;  
// C = alpha * A*B + beta * C    (Spalten-Major!)  
cublasSgemv(h, CUBLAS_OP_N, CUBLAS_OP_N,  
            N, N, N, &alpha, dA, N, dB, N, &beta, dC, N);  
cublasDestroy(h);
```

cuBLAS nutzt Tensor-Cores und architektur-spezifische Kachelung. Für *dichtes* GEMM ist ein selbst geschriebener Kernel selten konkurrenzfähig – die eigene Implementierung dient dem *Verständnis*.

Debugging & Profiling

GPGPU / CUDA – Vorlesung 2



Debugging-Werkzeuge

- Mit `-g -G` für Device-Debug-Symbole übersetzen.
- `cuda-gdb` – ein CUDA-fähiger GNU-Debugger (Breakpoints, Einzelschritt, Inspektion des Zustands pro Thread).
- `compute-sanitizer` (früher `cuda-memcheck`) – erkennt Zugriffe außerhalb der Grenzen, fehlausgerichtete Zugriffe und Race Conditions.
- Beachten: ein Kernel mit fehlerhaftem Speicherzugriff kann still zurückkehren – *stets* `cudaGetLastError()` prüfen.



Profiling-Werkzeuge

- *Nsight Systems* (`nsys`) – systemweiter Zeitstrahl: wo wird die Zeit verbraucht (Kopien vs. Kernel vs. CPU)?
- *Nsight Compute* (`ncu`) – Tiefenanalyse pro Kernel: Occupancy, Speicherdurchsatz, Stall-Gründe, Roofline.
- Achten auf: divergente Sprünge, unkoaleszierte Zugriffe, niedrige Occupancy, Bankkonflikte, Overhead der Host-Device-Übertragung.
- `nvprof/nvvp` ist auf neueren GPUs durch Nsight ersetzt.



Eine Optimierungs-Checkliste

- 1 Ist der Kernel überhaupt der Engpass? (Zuerst profilieren – `nsys`.)
- 2 Sind die globalen Zugriffe *coalesced*?
- 3 Werden wiederverwendbare Daten im *Shared Memory* eingelagert? Gibt es *Bankkonflikte*?
- 4 Gibt es *Sprungdivergenz* innerhalb von Warps?
- 5 Ist die *Occupancy* hoch genug, um Latenz zu verdecken?
- 6 Speicher- oder rechengebunden? (Roofline – `ncu`.)
- 7 Überlappen Kopien und Rechnen (*Streams*)?
- 8 Könnte eine *Bibliothek* (cuBLAS/Thrust) es besser?

Zusammenfassung

GPGPU / CUDA – Vorlesung 2



Kernaussagen

- Zuerst *messen* (Roofline, Profiler), dann gezielt optimieren.
- *Coalescing* und richtige *Datenlayout* (SoA) sichern Bandbreite.
- Shared Memory verwandelt speichergebundene Kernel in rechengebundene, indem es *Datenwiederverwendung* ermöglicht – auf *Bankkonflikte* achten (Padding).
- *Occupancy* verdeckt Latenz, ist aber Mittel, nicht Zweck.
- *Divergenz* innerhalb eines Warps serialisiert die Ausführung.
- Warp-Primitive und Atomics ermöglichen effiziente Kooperation.
- *Streams* überlappen Transfer und Rechnung; *Bibliotheken* schlagen oft den eigenen Kernel.